

React + PHP: The Thin Stack

Il protocollo miniCMS per Web App moderne

Simone Pizzi

Terza Edizione

RUNTIME EDIZIONI

React + PHP: The Thin Stack
Il protocollo miniCMS per Web App moderne
Terza Edizione — Giugno 2026

© 2026 Simone Pizzi
© 2026 Runtime Edizioni

Tutti i diritti riservati. Nessuna parte di questo libro può essere riprodotta senza il consenso scritto dell'editore, salvo brevi citazioni in recensioni.

I marchi e i nomi di prodotto citati sono dei rispettivi proprietari.
www.runtimeradio.com

[Testo della dedica da confermare]

placeholder allineato a destra, in corsivo,
come da tua indicazione

P A R T E I

La Visione

Il perché. La filosofia che guida ogni decisione tecnica.

CAPITOLO 1: Manifesto

A Valerio Galano, perché lui è come Neo, vede mondi nel codice e mi ha insegnato a ragionare in un modo diverso. Chissà se un po' anche io gli ho insegnato qualcosa.

A Giuseppe Pugliese che, anche se vede il mondo in un modo tutto suo, ritiene sia un orgoglio essere uno sviluppatore web e persevera nella sua arte con passione.

*
**

Perché Esiste Questo Protocollo

Esiste una tensione irrisolta al centro dello sviluppo web moderno.

Da un lato, il frontend ha raggiunto una maturità estetica e funzionale straordinaria: React, TypeScript, Tailwind. Animazioni fluide, componenti riutilizzabili, type safety, hot reload. L'esperienza di sviluppo è diventata un piacere, e il prodotto finale, quando fatto bene, è visivamente e funzionalmente superiore a qualsiasi soluzione del passato.

Dall'altro lato, questa rivoluzione ha portato con sé una complessità infrastrutturale sproporzionata rispetto ai bisogni reali della maggioranza dei siti. Node.js, database cloud, container, pipeline CI/CD, micro-servizi, CMS headless con piani in abbonamento. L'overhead tecnico e il costo operativo sono diventati la norma anche per siti che potrebbero girare perfettamente su un hosting da cinque euro al mese.

Questo protocollo nasce da una domanda precisa: **è possibile avere la potenza estetica e tecnica di React senza abbandonare la semplicità, il controllo e l'economicità di un backend PHP con SQLite?**

La risposta, costruita su mesi di lavoro reale su progetti reali, è sì.

*
**

Il Principio Fondativo: La Separazione dei Piani

Il Modello Universale non è una tecnologia. È un'architettura mentale.

Separa con nettezza due piani che spesso vengono confusi:

Il Piano della Presentazione appartiene a React. È il luogo della forma, dell'interazione, dell'animazione, della tipografia, della palette colori. È dove vive il talento estetico, dove Tailwind traduce l'intenzione visiva in CSS preciso, dove fra-

mer-motion aggiunge peso e respiro ai movimenti. Questo piano è compilato, ottimizzato, servito come asset statico.

Il Piano dei Dati appartiene a PHP e SQLite (o MySQL quando necessario). È il luogo della persistenza, della logica di business, della sicurezza, del ciclo di vita dei contenuti. Non è «il backend» nel senso pesante del termine: nessun framework, nessun ORM, nessuna dipendenza esterna. Solo PHP nativo, PDO, e un database file-based che non richiede configurazione server.

Questi due piani comunicano attraverso un contratto preciso: le API REST. Il frontend non sa niente del database. Il backend non sa niente di React. La loro separazione è la fonte di tutta la scalabilità e la manutenibilità del sistema.

✱
✱ ✱

La Scala, non l'Assenza

«Thin stack» non significa «backend assente». Significa un backend ridotto all'osso ma vero, e significa soprattutto una scala. Lo stesso scheletro (PHP nativo, un singleton PDO per richiesta, un file per endpoint, niente framework) si declina su gradini diversi a seconda di cosa serve. Al grado-zero c'è SQLite, un database che è un singolo file, senza un server da configurare né un segreto da custodire. Un gradino sopra c'è MySQL essenziale, quando i dati o il traffico lo richiedono. Più su ancora c'è MySQL ingegnerizzato, con connessioni rinforzate, prelude condivisi e scaffolding dedicato. Non sono tre architetture diverse: sono lo stesso modello a tre altezze.

Questo manuale è costruito su quattro siti reali che occupano punti diversi di quella scala, e li legge proprio così. Quando un capitolo mostra «tre modi di fare la stessa cosa», non sta elencando opzioni a caso: sta misurando quanto si può togliere, o aggiungere, allo stesso scheletro prima che cambi natura. La scala a tre gradini, dal grado-zero all'ingegnerizzato, è la chiave di lettura del libro intero.

✱
✱ ✱

Cosa Non È Questo Protocollo

Non è un framework. Non impone strutture di codice rigide, non richiede dipendenze specifiche, non vincola le scelte stilistiche.

Non è un CMS tradizionale. Non c'è un'interfaccia visual builder, non ci sono temi preconfezionati, non c'è un marketplace di plugin. Ogni sito costruito con questo protocollo è unico, fatto a mano, su misura per il suo scopo.

Non è una soluzione enterprise. Non è progettato per gestire milioni di utenti simultanei, flussi di dati complessi o architetture distribuite. È progettato per siti che devono essere eccellenti, veloci, sicuri e mantenibili da un team piccolo, o anche da una persona sola.

Non è per chi vuole un sito in dieci minuti. È per chi vuole capire cosa sta costruendo.

✱
✱ ✱

Quando NON Usare Questo Protocollo

Un manifesto onesto deve dire anche dove finisce. Il Thin Stack scambia la complessità dell'infrastruttura con la disciplina di chi lo scrive: toglie il framework e mette al suo posto la tua attenzione. Quando quel baratto non conviene, conviene un altro stack. Ecco i casi in cui sceglierei altro senza esitare.

Quando il team è grande. Le convenzioni qui non sono imposte da un framework, sono tenute insieme dalle persone. Con uno o due sviluppatori funziona; oltre i quattro o cinque, l'assenza di una struttura rigida diventa un costo, non una libertà, e un framework opinato (Laravel, Next.js con le sue regole) ripaga la curva di apprendimento.

Quando serve il tempo reale. Chat dal vivo, notifiche push istantanee, presenza, collaborazione simultanea su uno stesso documento: sono carichi che vogliono WebSockets e processi persistenti, non il modello richiesta-risposta di PHP su hosting condiviso. Si possono forzare, ma è la strada in salita.

Quando la scala è davvero alta. Decine di migliaia di richieste al secondo, picchi imprevedibili, necessità di scalare orizzontalmente su più nodi: qui servono code, cache distribuite e database gestiti. Il Capitolo 3 mette dei numeri concreti sotto questa frase, così la soglia non resta un'opinione.

Quando i dati sono complessi e molto relazionali. Transazioni distribuite, reportistica analitica pesante, modelli con decine di entità interconnesse: a un certo punto un ORM e un RDBMS ingegnerizzato non sono overhead, sono lo strumento giusto.

Quando la conformità è un requisito esplicito. Audit formali, certificazioni, ambienti regolati che pretendono framework e librerie con supporto commerciale e una catena di responsabilità documentata: il fai-da-te disciplinato, qui, è un rischio che non vale la pena correre.

La regola che tiene insieme questi casi è semplice: il Thin Stack è eccellente finché la complessità del problema sta sotto la complessità che un framework imporrebbe. Quando la supera, il framework smette di essere un peso e diventa una rete. Riconoscere quel punto di sorpasso è parte della stessa onestà tecnica che attraversa il resto del libro.

✱
✱ ✱

I Valori che Guidano Ogni Decisione

Controllo totale. Chi costruisce un sito con questo protocollo possiede ogni riga del suo stack. Nessun vendor lock-in, nessun aggiornamento forzato che rompe la produzione, nessuna dipendenza da un servizio esterno per la sopravvivenza del sito.

Leggerezza come principio, non come compromesso. SQLite non è la scelta «economica» rispetto a MySQL: è la scelta giusta per il 90% dei casi d'uso. La semplicità non è una limitazione da superare, è un obiettivo da raggiungere e difendere.

La sicurezza come architettura, non come patch. Le decisioni di sicurezza sono integrate nel design del sistema: database fuori dalla root pubblica, uno script di build

che non lascia mai il database nel deploy, sessioni PHP con cookie HttpOnly, password mai in chiaro. Non sono misure aggiunte dopo, sono la struttura stessa.

Più ingegnerizzato non vuol dire più sicuro. È la lezione più scomoda che questi siti insegnano, e ritorna in quasi ogni capitolo. Il sito tecnicamente più ricco, con le connessioni più blindate e l'infrastruttura più curata, è spesso quello con i fondamentali più fragili: una password di default scritta nel codice, nessun backup automatico, un argine anti-abuso che manca proprio dove servirebbe. Aggiungere complessità non paga interessi di sicurezza da sola, e a volte distrae dalle due righe che contano davvero. Diffidare dell'equazione «più strati uguale più sicuro» è uno dei fili che tengono insieme questo manuale.

La documentazione come parte del codice. Un sistema che non si capisce è un sistema che non si può mantenere. Questo protocollo è documentato con la stessa cura con cui è costruito, perché la conoscenza deve rimanere accessibile anche quando il contesto cambia.

L'esperienza reale come unico validatore. Ogni pattern documentato in questo manuale è stato estratto da codice che gira in produzione. Le lezioni più importanti vengono da incidenti veri, non da scenari ipotetici: il crash notturno del WAL che ha costretto Runtime Radio a migrare d'urgenza su MySQL, l'ondata di bot che ne ha saturato l'entry-point. La teoria senza la cicatrice non insegna abbastanza.

*
**

A Chi È Rivolto

A chiunque voglia costruire un sito web che sia una cosa viva: non un template, non un WordPress personalizzato, non un sito sputato fuori da un builder.

Al developer che conosce React e vuole un backend senza dover imparare un framework intero.

Al freelance che deve consegnare un sito veloce, mantenibile e sicuro a un cliente che non ha budget per infrastrutture cloud.

All'autore, al musicista, al festival, alla radio che vuole una presenza digitale propria, controllata, indipendente dai capricci delle piattaforme.

A chiunque creda che il web possa essere ancora un posto fatto da persone, per persone, senza intermediari.

*
**

Due Voci: «Dal vivo» e «Il Canone»

Questo manuale fa una cosa che la maggior parte dei testi tecnici evita: mostra il codice reale com'è, non come dovrebbe essere. È la sua forza, ma può confondere, perché in una stessa pagina convivono due cose diverse, e vanno tenute distinte.

La prima voce è «**dal vivo**»: la fotografia di cosa fanno davvero i quattro siti. Qui ci sono le scelte buone e anche le cicatrici, gli anti-pattern, le falle lasciate aperte. Quando

il testo racconta che un sito salva l'HTML grezzo senza sanitizzarlo, o espone uno script di migrazione, non lo sta raccomandando: lo sta documentando. È il corpo di ogni capitolo, ed è scritto senza sconti proprio perché la cicatrice insegna più della teoria.

La seconda voce è «**il Canone**»: la regola da seguire, distillata. Ogni capitolo si chiude con un riquadro intitolato **Il Canone**, che separa nettamente la prescrizione dalla fotografia. Lì trovi cosa fare in un progetto nuovo, ripulito dalle imperfezioni dei casi reali. Se hai dieci secondi e vuoi solo la norma, leggi quel riquadro; se vuoi capire *perché* la norma è quella, leggi il capitolo che lo precede.

NOTA

La regola di lettura, in una riga Il corpo del capitolo dice cosa il codice *fa* («dal vivo»); il riquadro finale dice cosa tu *dovresti* fare («il Canone»). Quando i due divergono, ha sempre ragione il Canone: la divergenza è il punto, non un errore di stampa.

**

Come Usare Questo Manuale

Il manuale è organizzato in capitoli tematici indipendenti. Non è necessario leggerlo dall'inizio alla fine: ogni capitolo è una reference autonoma.

Per iniziare un nuovo progetto da zero, la **BOILERPLATE-CHECKLIST** è il punto di partenza pratico.

Per migliorare un progetto esistente, i capitoli specifici (Database Strategy, Security & Auth, SEO Pre-rendering) offrono pattern applicabili in modo chirurgico.

Per imparare dalla storia, i capitoli con la voce esperienziale (il crash del WAL, l'attacco dei bot su Runtime Radio, la migrazione a MySQL) sono la lettura più onesta che questo manuale può offrire.

Il codice non mente. Le cicatrici nemmeno.

**

IMPORTANTE

Il Canone

- Separa i due piani: React compilato per la presentazione, PHP nativo con PDO per i dati, un contratto REST tra loro.
- Scegli il gradino giusto della scala (SQLite grado-zero → MySQL essenziale → MySQL ingegnerizzato), mai più di quanto serve.
- Tratta la sicurezza come architettura, non come patch, e diffida dell'equazione «più strati uguale più sicuro».
- Documenta com'è il codice, non com'è bello: la cicatrice insegna più della teoria.

- Usa il Thin Stack finché la complessità del problema resta sotto quella che un framework imporrebbe; superato quel punto, scegli il framework.

*
**

«La perfezione si raggiunge non quando non c'è più niente da aggiungere, ma quando non c'è più niente da togliere.» Antoine de Saint-Exupéry

*
**

Prossimo Capitolo: Architettura e Struttura Progetto. Dove le idee diventano cartelle.

CAPITOLO 10: Security & Auth

La sicurezza è l'unica lente di questo libro in cui i tre siti non scalano in parallelo con l'ingegnerizzazione del backend. Negli altri capitoli vale una regola intuitiva: più un sito è cresciuto, più la sua infrastruttura è ricca. Qui no. Lo stesso scheletro thin-stack (PHP nativo, sessione su cookie, regole Apache nei `.htaccess`) regge tre gradini di difesa decrescenti, ma il gradino più alto non è quello del sito col backend più sofisticato.

SimonePizziWebSite (SPW) ha il perimetro più maturo: cookie `Secure`, anti session-fixation, invalidazione globale delle sessioni, recupero password completo, IP anti-spoofing. SitoRuntime (SR), il sito più ingegnerizzato del trio, quello delle cicatrici di scalabilità, è solo a metà strada: ha ruoli e token CSRF, ma il cookie viaggia senza `Secure`, non rigenera la sessione al login e il suo rate-limit si aggira con un header. DISINTELLIGENZA (DIS) è l'autenticazione di grado zero: niente CSRF, niente rate-limit, cookie ai valori di default di PHP. Eppure è l'unico dei tre a portare due idee di sicurezza proprie e di valore, la difesa anti-frode di un'azione pubblica (il voto del festival) e il backup automatico prima di un'operazione distruttiva.

È la dimostrazione più netta della tesi che attraversa tutto il libro: più ingegnerizzato non significa più sicuro, e più sicuro non significa più completo su ogni singolo punto. Il modo giusto di leggere ogni difesa è come una scala di sottrazione: cosa resta, e cosa si rompe, quando togli un flag dal cookie, un token da una richiesta, un contatore da un endpoint di login.

NOTA

Una nota di metodo. Tutti gli stralci di codice di questo capitolo vengono dallo stato reale dei tre siti, citati come `file:linea`. Quando il capitolo dice «il Modello raccomanda» sta prescrivendo; quando dice «SPW fa» o «SR fa» o «DIS fa» sta fotografando il codice in produzione. Le due cose non sempre coincidono, ed è proprio in quello scarto che si impara.

**

1. Il perimetro comune: sicurezza fatta a mano

Prima delle divergenze conviene fissare ciò che i tre siti condividono. Sotto le tre implementazioni c'è lo stesso oggetto, con cinque tratti ricorrenti.

1) Nessuna libreria di autenticazione. Niente Passport, niente pacchetto JWT, nessun framework di sessione. Tutto poggia su primitive PHP native: `session_*`, `password_hash/password_verify`, `random_bytes`, `hash_equals`, e sulle regole Apache nei `.htaccess`.

L'auth è fatta a mano, ed è la scelta fondante che rende ogni differenza tra i siti una scelta visibile, non una configurazione sepolta dentro un vendor.

2) Sessione su cookie più password_verify: lo zoccolo identico. Tutti e tre aprono una sessione PHP nativa, cercano l'utente per username, verificano la password contro un hash PASSWORD_DEFAULT, e popolano \$_SESSION. Il sistema non conosce mai la password in chiaro:

```
// Lo schema comune ai tre siti: nessun segreto in chiaro
$hash = password_hash($password, PASSWORD_DEFAULT); // alla creazione
// ...
if (password_verify($input, $user['password_hash'])) { /* login ok */ }
```

3) Il gate è una manciata di righe in cima all'endpoint. Non c'è middleware né router: la protezione di un endpoint sono poche righe all'inizio del ramo mutativo, che pretendono una sessione valida prima di toccare i dati. È la versione thin-stack del middleware. Ma quanto fa quella manciata di righe (solo sessione? più CSRF? più ruolo? più invalidazione?) è il primo grande asse di divergenza, e lo vediamo al §2.

4) Difesa «JSON-first» anche sugli errori di sicurezza. Un accesso negato non produce una pagina di errore di Apache: si imposta il codice HTTP semanticamente corretto (401/403/429) e si risponde con un oggetto {status|success, message|error}. Il frontend reagisce per codice.

5) Hardening a livello server via .htaccess. Tutti hanno almeno un .htaccess che imposta header di sicurezza e nega l'accesso a file sensibili. È il secondo strato, fuori dall'applicazione, e anche qui la copertura va dal completo (HTTPS forzato, HSTS, CSP, PHP-off) al minimo (deny di due estensioni).

A questi si aggiunge un tratto negativo condiviso: nessuno dei tre tiene un audit-log degli accessi, e in due casi su tre il contatore brute-force del login non vive nemmeno nel database. Il perimetro è sottile per costruzione. La domanda del capitolo è quanto sottile si può andare prima che qualcosa si rompa.

*
**

2. Il gate: middleware unico, gate componibile, gate inline

Proteggere un endpoint significa decidere, in cima al codice, se chi chiama ha diritto di proseguire. I tre siti risolvono lo stesso problema con tre architetture diverse, ed è il primo punto in cui la scala si fa visibile.

2.1 SPW: il gate unico Auth::check()

SPW concentra tutto in una classe inclusa da ogni endpoint protetto. Una sola riga, Auth::check(), fa tre cose in sequenza: esige una sessione valida, verifica l'anti-CSRF sui metodi mutativi, controlla l'invalidazione via session_version.

```
// public/api/articles.php:238-239 - il consumatore tipico
elseif ($method === 'POST') {
    Auth::check(); // sessione + CSRF + session_version, tutto qui
    // ... da qui in poi si può scrivere
```

Il vantaggio è che la copertura è automatica: ogni nuovo endpoint che include `auth_helper.php` e chiama `Auth::check()` eredita tutte le difese insieme. Non c'è modo di ricordarsi la sessione e dimenticare il CSRF.

2.2 SR: il gate componibile a funzioni

SR scompone la stessa protezione in mattoni separati, autorizzazione da una parte e anti-CSRF dall'altra, che ogni endpoint compone a mano nell'ordine giusto:

```
// public/api/auth_utils.php - i mattoni
function isLoggedIn() { return isset($_SESSION['user_id']); }
function isAdmin()    { return isLoggedIn() && ($_SESSION['role'] ?? '') ===
↳ 'admin'; }

// public/api/upload.php:8-13 - il consumatore li compone
if (!isLoggedIn()) { http_response_code(401); echo json_encode([...]); exit;
↳ }
// ...
validateCsrf();
```

È più flessibile, e il gate a ruolo (`isAdmin()`) è gratis: arriva quel `admin/editor` che SPW non ha. Ma è anche più facile da dimenticare, perché la protezione dipende dalla disciplina del singolo endpoint.

2.3 DIS: il gate inline grezzo

DIS porta lo stesso schema all'estremo. Niente file di funzioni-mattone: il gate è ricostruito a mano in ogni ramo come un `isset()` nudo.

```
// public/api/reset_votes.php:11 - il gate per intero, inline
if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    http_response_code(401); die(...);
}
```

ATTENZIONE

Middleware o disciplina? Il gate che si dimentica Il gate componibile e quello inline hanno un rischio strutturale che il gate unico non ha: la sicurezza dipende dall'ordine dei rami e dalla memoria di chi scrive. In DIS i rami `participants.php?update_status` e `update_round` sono protetti dal solo `isset($_SESSION['user_id'])`, non da `isAdmin()`. Risultato: un editor, non un amministratore, può approvare o respingere partecipanti, mandare le email e spostare i round del festival. La stessa crepa si vede in SR, dove `list_users` e `create_user` stanno prima del blocco 401 e si autoprotettono solo con il check di ruolo locale. Funziona finché tutti i rami sono scritti con disciplina. Ma un gate unico (`Auth::check()`) non può essere dimenticato su un endpoint: o lo includi, o l'endpoint non parte. È questa la differenza vera tra middleware e convenzione. Le conseguenze sul festival si vedono al CAP 18.

✱
✱ ✱

3. CSRF: tre gradini di difesa

Il Cross-Site Request Forgery è il problema di impedire che un sito terzo invii richieste mutative sfruttando il cookie di sessione che il browser dell'utente allega in auto-

matico. È la difesa portante di questo capitolo, e i tre siti la risolvono su tre livelli distinti.

3.1 SPW: controllo `Origin/Referer` (server-side, zero handshake)

SPW non usa token: dentro `Auth::check()`, sui metodi non-safe, confronta l'host di `Origin/Referer` con quello di `SITE_URL`.

```
// public/api/auth_helper.php:21-37 (estratto)
$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
if (!in_array($method, ['GET', 'HEAD', 'OPTIONS'], true)) {
    $source = $_SERVER['HTTP_ORIGIN'] ?? $_SERVER['HTTP_REFERER'] ?? '';
    if ($source !== '') {
        $sourceHost = parse_url($source, PHP_URL_HOST);
        $allowedHost = parse_url(SITE_URL, PHP_URL_HOST);
        $isLocalDev = in_array($sourceHost, ['localhost', '127.0.0.1'], true);
        if ($sourceHost !== $allowedHost && !$isLocalDev) {
            http_response_code(403);
            echo json_encode(['status' => 'error', 'message' => 'Origine della richiesta non valida']);
            exit;
        }
    }
}
```

Non richiede alcun handshake col client, perché il browser invia `Origin/Referer` da solo. E vivendo dentro `Auth::check()`, copre in automatico ogni endpoint che include la guardia.

3.2 SR: token sincronizzato X-CSRF-Token

SR adotta il pattern da manuale: un token casuale per sessione, restituito al client al login, rispedito dal client in un header e validato con `hash_equals` (confronto timing-safe).

```
// public/api/auth_utils.php:20-35
function generateCsrfToken(): string {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function validateCsrf(): void {
    $incoming = $_SERVER['HTTP_X_CSRF_TOKEN'] ?? '';
    $stored = $_SESSION['csrf_token'] ?? '';
    if (!$stored || !hash_equals($stored, $incoming)) {
        http_response_code(403);
        echo json_encode(['success' => false, 'error' => 'Token di sicurezza non valido. Ricarica la pagina.']);
        exit;
    }
}
```

3.3 DIS: niente

In DIS le mutazioni sono protette dal solo cookie di sessione. Un grep su tutta `public/api/` non trova nessun token CSRF, nessun check `Origin/Referer`: la difesa è assente.

ATTENZIONE

CSRF nel thin stack: Origin/Referer, token sincronizzato, o niente C'è un sotto-testo controintuitivo in questi tre gradini. La soluzione più «da manuale», il token

sincronizzato di SR, è anche la più facile da indebolire, perché dipende dalla disciplina per-ramo: basta un endpoint mutativo che dimentichi `validateCsrf()` e il buco è aperto. La soluzione più semplice, il check `Origin/Referer` di SPW, copre di più proprio perché è centralizzata dentro il gate unico. Più codice non vuol dire più copertura. È la tesi di fondo del libro applicata al CSRF.

**

4. Il cookie di sessione e l'anti session-fixation

NOTA

Un errore comune sui cookie di sessione. Verrebbe da prescrivere un cookie CON `cookie_secure = 1` e `cookie_samesite = 'Lax'`. Guardando il codice reale, due cose vanno corrette: il `SameSite` giusto, e quello che i siti usano davvero, è `Strict`, non `Lax` (sia SPW sia SR); e solo SPW imposta davvero tutti e tre i flag.

I flag vanno impostati prima di `session_start()`, altrimenti non hanno effetto sul cookie corrente. È qui che la scala si vede meglio.

SPW imposta i tre flag in un file condiviso:

```
// public/api/auth_helper.php:7-11
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_secure', 1);
ini_set('session.cookie_samesite', 'Strict');
session_start();
```

SR omette `Secure`:

```
// public/api/auth_utils.php:4-9
ini_set('session.cookie_httponly', '1');
ini_set('session.cookie_samesite', 'Strict');
session_start(); // NB: cookie_secure NON impostato
```

DIS non imposta nessun flag: il comportamento del cookie dipende interamente dal `php.ini` dell'hosting. Una dipendenza implicita e silenziosa.

A questo si aggiunge l'anti session-fixation, cioè rigenerare l'ID di sessione subito dopo il login per invalidare un ID eventualmente piantato prima dall'attaccante. Solo SPW lo fa:

```
// public/api/auth.php:186-188 (estratto del ramo di successo)
if ($user && password_verify($password, $user['password_hash'])) {
    session_regenerate_id(true); // anti session-fixation: una riga, spe
↳ sso assente
    $_SESSION['user_id'] = $user['id'];
    // ...
}
```

SR e DIS non rigenerano: un ID di sessione fissato prima del login resta valido dopo.

ATTENZIONE

I flag del cookie, e perché HSTS non è il redirect HTTPS Togliere `Secure` non è un dettaglio cosmetico. Senza `Secure`, il cookie di sessione può viaggiare anche su

HTTP in chiaro. Si potrebbe pensare che HSTS protegga comunque, ma HSTS protegge solo dopo la prima visita HTTPS riuscita. SR applica HSTS ma non forza il redirect 301 a HTTPS: c'è una finestra reale di esposizione su HTTP al primo accesso. SPW chiude la finestra da entrambi i lati, con `cookie_secure=1` e con `RewriteCond %{HTTPS} !=on → 301`. La lezione: Secure sul cookie e redirect HTTPS forzato sono due difese distinte; HSTS non sostituisce nessuna delle due. Curiosità storica (SPW): fino alla v1.18 i flag stavano solo in `auth.php`, così un cookie emesso da un altro endpoint nasceva debole. La v1.19.0 ha spostato gli `ini_set` nel file condiviso `auth_helper.php`. La config di sicurezza della sessione va nel file incluso da tutti, non nell'endpoint di login.

✱
✱ ✱

5. check_auth e lo stato di sessione

Il frontend non deve mai conservare la password o lo stato di login in `localStorage`: interroga il server per sapere se la sessione è ancora valida.

```
// schema di check_auth (SPW/DIS)
if (isset($_SESSION['user_id'])) {
    echo json_encode([
        'status' => 'success',
        'user'   => ['username' => $_SESSION['username'], 'role' => $_SESSION
    < N['role']]
    ]);
}
```

NOTA

Lo username non è garantito in sessione. Verrebbe da leggere `$_SESSION['username']` come se fosse sempre presente. In SR non lo è: il login salva `user_id` e `role` ma non `username` (`admin.php:123-124`). La conseguenza è concreta: `check_auth` restituisce `username: null`, e il salvataggio articolo ripiega su `author = $_SESSION['username'] ?? 'Admin'`. In SPW e DIS lo `username` è in sessione e l'esempio regge. Non è un buco di sicurezza, ma un'incoerenza di stato reale: lo stesso campo non è garantito su tutti e tre i siti.

✱
✱ ✱

6. Le password e la difesa brute-force

L'hashing è l'unico punto in cui i tre siti sono identici e tutti corretti:

`password_hash($pass, PASSWORD_DEFAULT)` alla creazione, `password_verify` alla verifica. Il sistema non conosce mai la password in chiaro, e l'algoritmo può evolvere (da `bcrypt` ad `argon2`) senza che si tocchi una riga di codice.

NOTA

La difesa brute-force non è un `sleep(1)`. Verrebbe da ridurre la difesa brute-force a un `sleep(1)`, ma è una soluzione incompleta e tarata sul solo SR. Il `sleep(1)` è ap-

pena un accorgimento; la difesa vera è il logout, e soprattutto conta da quale IP lo si misura.

La domanda non è «come rallento i tentativi» ma dove vive il contatore. Tre risposte. SPW lo tiene in una tabella DB `login_attempts`: dopo 5 tentativi falliti da un IP in 15 minuti scatta il 429, e al login riuscito il contatore si azzerà.

```
// public/api/auth.php:177-211 (estratto)
if ($attempts >= 5) {
    http_response_code(429);
    echo json_encode(['status' => 'error', 'message' => 'Too many failed log
↳ in attempts. Try again in 15 minutes.']);
    exit;
}
// ... su fallimento:
$pdo->prepare("INSERT INTO login_attempts (ip_address) VALUES (?)")->execute
↳ ([$ip_address]);
```

SR lo tiene su file: un JSON per IP in `.cache/ratelimit/<md5(ip)>.json`, finestra 900 secondi, soglia 5, più `sleep(1)` sul fallimento. Niente tabella DB, coerente con lo schema MySQL che non contiene `login_attempts`. DIS non lo tiene da nessuna parte: il login si può martellare quanto si vuole.

CONSIGLIO

Il contatore brute-force: file, DB, o assenza Non c'è una sede giusta in assoluto.

La tabella DB di SPW è transazionale e si presta al riuso (lo stesso `login_attempts` serve anche il recovery). Il file di SR evita di gravare sul database ma vive fuori dallo schema, quindi non compare in un dump né in una migrazione: è invisibile finché non lo cerchi nel filesystem. L'assenza di DIS è coerente con un sito-festival dove l'admin è uno solo e l'attrito su un login interno è basso, ma resta un'assenza, non una scelta documentata.

Il punto più sottile, però, è da quale variabile leggi l'IP. È il box-ancora di questo capitolo.

ATTENZIONE

Fidarsi dell'IP: il rate-limit che non limita (*box-ancora, richiamato al CAP 18 e al CAP 20*) Un logout «5 tentativi per IP» vale esattamente quanto la tua capacità di sapere qual è l'IP. E qui i tre siti divergono in modo istruttivo.

SR si fida dell'header sbagliato. Prende l'IP da `X-Forwarded-For`, per primo e senza validazione:

```
// public/api/admin.php:106-107
$ip = $_SERVER['HTTP_X_FORWARDED_FOR'] ?? $_SERVER['REMOTE_ADDR'] ?? 'unknown
↳ n';
$ip = trim(explode(',', $ip)[0]); // ← l'attaccante controlla questo header
↳ r
```

Quell'header è scritto dal client. Un attaccante lo varia a ogni richiesta e il contatore non si incrementa mai: il logout 5/15min si aggira.

SPW si fida dell'IP TCP, e dell'header solo dietro proxy interno. La funzione `getClientIp()` usa `REMOTE_ADDR` se è già pubblico, e accetta `X-Forwarded-For` (validato `NO_PRIV_RANGE|NO_RES_RANGE`) solo quando `REMOTE_ADDR` è privato, cioè solo dietro un proxy fidato. Niente spoofing.

DIS usa `REMOTE_ADDR` grezzo, e qui è un pregio. Per la barriera anti-doppio-voto (§12), `REMOTE_ADDR` non è falsificabile a livello TCP, quindi la barriera regge. Il rovescio è il NAT: dietro un proxy o una rete condivisa, molti utenti collassano sullo stesso IP.

La lezione non è «usa sempre `REMOTE_ADDR`». È che l'IP giusto dipende dal modello d'abuso. Un login da forzare (vuoi che l'attaccante non possa cambiare la propria identità, quindi diffidi di `X-Forwarded-For`) è il problema opposto di un voto pubblico da non duplicare; eppure la conclusione è la stessa: l'header controllato dal client non è affidabile. Lo stesso `REMOTE_ADDR` grezzo è un buco in un contesto e una difesa nell'altro.

**

7. session_version: invalidare le sessioni a costo zero

C'è un problema classico dell'auth su sessione: come disconnetti tutte le sessioni di un utente, per esempio dopo un reset password, se non tieni uno store server-side delle sessioni attive? Solo SPW risolve, con un trucco a costo zero: un intero `session_version` in `users`, copiato in `$_SESSION` al login e confrontato a ogni richiesta protetta.

```
// public/api/auth_helper.php:51-57 - dentro Auth::check()
try {
    $stmt = $pdo->prepare("SELECT session_version FROM users WHERE id = ?");
    $stmt->execute([$$_SESSION['user_id']]);
    $row = $stmt->fetch();
    if (!$row || (int)$row['session_version'] !== (int)$_SESSION['session_v
↵ ersion'] ?? -1) {
        session_destroy();
        http_response_code(401);
        echo json_encode(['status' => 'error', 'message' => 'Sessione scadut
↵  a. Effettua nuovamente il login.']);
        exit;
    }
} catch (PDOException $e) {
    error_log('session_version check failed: ' . $e->getMessage());
    http_response_code(401); // ← fail-closed: in caso di errore DB, NEGA
    echo json_encode(['status' => 'error', 'message' => 'Sessione non verifi
↵  cabile. Riprova.']);
    exit;
}
```

Basta incrementare `session_version` di un numero (lo fa il reset password, §8) e tutte le sessioni col numero vecchio diventano invalide al primo controllo. Logout-everywhere senza alcuno store di sessioni.

CONSIGLIO

Fail-closed, non fail-open Il dettaglio che distingue una difesa fatta bene è il ramo catch. Se il controllo di `session_version` solleva una `PDOException` (DB irraggiungibile), SPW nega l'accesso (401), non lo concede. La regola: quando una verifica di sicurezza non può essere completata, l'esito di default deve essere «negato». SR e DIS non hanno questo meccanismo, e in SR un cambio password non disconnette le altre sessioni; in DIS nemmeno.

✱
✱ ✱

8. Recovery e reset password fatti bene

È un'intera sezione che il capitolo prima non aveva, perché solo SPW implementa il recupero password self-service. SR e DIS hanno solo `change_password` autenticato, dove devi già essere dentro: password dimenticata uguale nessun rientro.

Il flusso di SPW ha quattro accortezze che vale la pena guardare una per una.

La prima è il token monouso casuale, con scadenza.

```
// public/api/auth.php:98-106
$token = bin2hex(random_bytes(32)); // 32 byte casuali
$expires = date('Y-m-d H:i:s', strtotime('+1 hour'));
$pdo->prepare("DELETE FROM password_resets WHERE user_id = ?")->execute([$user_id]); // invalida i precedenti
$pdo->prepare("INSERT INTO password_resets (user_id, token, expires_at) VALUES (?, ?, ?)")->execute([$user_id, $token, $expires]);
```

La seconda è il link costruito da URL canonico, non da `HTTP_HOST`. È la difesa contro il *password-reset poisoning*: se il link nell'email si costruisse da `HTTP_HOST` (controllabile con un header `Host` falsificato), l'attaccante potrebbe far recapitare alla vittima un link che punta al suo dominio e intercettare il token.

```
// public/api/auth.php:227-230
$link = SITE_URL . "/admin/reset-password/{$token}"; // SITE_URL hardcoded
// mai HTTP_HOST
```

La terza è l'essere enumeration-safe. La richiesta di recupero risponde sempre con lo stesso messaggio generico («se l'account esiste, riceverai un'email»), che l'utente esista o no, così non si può usare il form per scoprire quali email sono registrate. Lo stesso contatore `login_attempts` viene riusato qui con namespacing della chiave (`'rec:' + sha256(IP)`), per limitare anche gli abusi del recovery.

La quarta è che il reset invalida le sessioni. Applicando la nuova password, `session_version` viene incrementato, e per quanto visto al §7 tutte le sessioni aperte cadono:

```
// public/api/auth.php:127-149 (estratto)
if (strlen($newPassword) < 12) { /* 400: "almeno 12 caratteri" */ }
// ... verifica token non scaduto ...
$hash = password_hash($newPassword, PASSWORD_DEFAULT);
$pdo->prepare("UPDATE users SET password_hash = ?, session_version = session_version + 1 WHERE id = ?")->execute([$hash, $reset['user_id']]);
```

```
$pdo->prepare("DELETE FROM password_resets WHERE token = ?")->execute([$token]); // token monouso
```

NOTA

Lo schema che non c'è. La tabella `password_resets` è usata ovunque in `auth.php`, ma nessun file la crea: lo script di migrazione che la generò è stato cancellato dopo l'esecuzione in produzione. Lo stesso vale per la colonna `session_version`, aggiunta a caldo con un `ALTER TABLE` idempotente dentro `auth.php`. È un debito di tracciabilità: lo schema vero non è ricostruibile da un singolo file. Il tema delle migrazioni *usa-e-getta* torna al CAP 15 (Database Evolution).

*
**

9. Credenziali di default: random, hardcoded, omessa

Il primo amministratore è un problema di sicurezza spesso sottovalutato. I tre siti lo risolvono in tre modi, e qui si chiude un punto anticipato al CAP 5 (Backend Logic).

ATTENZIONE

Il seeding dell'admin: cosa NON fare SPW la genera random e la stampa una volta. La password del primo admin è casuale e mostrata una sola volta in fase di setup. È la scelta corretta. SR la hardcode a `runtime2026`. Il file di manutenzione `fix_users_table.php` ricrea l'utente admin con `password_hash('runtime2026', ...)` se la tabella è vuota, e quella password è committata nel repo. La mitigazione reale è il `.htaccess`, che nega l'accesso HTTP a `fix_users_table.php` (§10); ma non esiste alcun flusso che obblighi a cambiare la default, e se nessuno usa `change_password` l'account admin resta indovinabile. DIS la omette del tutto. Non c'è nessun seeding: `init_db.php` ha eliso la creazione dell'admin, e `users.php` può creare utenti solo se sei già admin. Risultato: niente default indovinabile (il vantaggio), ma il primo admin vive solo nel file `.sqlite` e non è ricostruibile dal repo (lo svantaggio, con il classico problema dell'uovo e la gallina al primo deploy pulito). Tre posizioni sulla stessa scala: la random è la più sicura, l'omessa è la più spartana ma non insicura, l'hardcoded è l'unica davvero pericolosa. E non perché la password esista, ma perché niente costringe a cambiarla.

*
**

10. Proteggere il database-a-file e gli script di manutenzione

Quando il database è un file (`.sqlite`), o quando nel docroot vivono script potenti (migrazioni, fix, reset), il `.htaccess` diventa parte del perimetro di sicurezza.

Il primo compito è negare l'accesso diretto al DB-a-file. DIS tiene il `.sqlite` in una cartella `.data/` generata a runtime e protetta:

```
# DIS: deny dei file di dati e backup  
<FilesMatch "\.(sqlite|bak)$">
```

```
Require all denied
</FilesMatch>
```

La regola di base è quella che il capitolo già insegnava (file fuori dalla root pubblica, oppure `Require all denied` più nomi non prevedibili). La novità è che DIS la accoppia a una cartella `.data/` creata a runtime dal bootstrap, di cui parliamo al CAP 5.

Il secondo compito è negare gli script di manutenzione, ed è qui che SR ha il `.htaccess` più interessante dei tre, perché blocca un'intera famiglia di file per prefisso, prima ancora che PHP li veda:

```
# public/.htaccess:80-81 (SR)
<FilesMatch "^(debug_|test_|emergency_|migrate_|fix_|init_|rebuild_|setup_|o
↳ ptimize_)">
    Order allow,deny
    Deny from all
</FilesMatch>
# blocca anche gli URL tipici degli scanner, prima di PHP
RewriteRule ^(wp-|wordpress|xmlrpc|\.env|\.git|cgi-bin|phpmyadmin) - [R=404,
↳ L]
```

È questo `deny` a rendere non eseguibile via browser la `fix_users_table.php` con la password `runtime2026` del §9.

Il terzo compito riguarda gli upload. Se un attaccante riesce a caricare un `.php`, l'esecuzione va spenta a livello di cartella, come fa SPW:

```
# public/uploads/.htaccess (SPW)
<IfModule mod_php.c>
    php_flag engine off
</IfModule>
<FilesMatch "\.(php|phtml|php[0-9]|phps|cgi|pl|py|sh)$">
    Require all denied
</FilesMatch>
```

ATTENZIONE

Il deny che manca dove serve di più Attenzione a non leggere queste regole come una checklist uniforme: la copertura è disomogenea. SR ha il `deny` by-prefix più sofisticato dei tre, ma non ha un `uploads/.htaccess` che spenga PHP nella cartella di caricamento. DIS protegge il `.sqlite` e i `.bak`, ma i suoi script `update_db_*` non rientrano in nessun pattern di `deny` e restano raggiungibili. Ogni sito ha blindato la porta che aveva visto, lasciandone aperta un'altra. La protezione dell'upload pubblico, e la catena d'abuso che ne nasce in DIS, è il cuore del CAP 7.

**

11. Errori parlanti per l'utente, opachi per l'attaccante

Un errore PHP non gestito che finisce nell'output può rivelare percorsi, query, struttura del database (è il cosiddetto *path/information disclosure*). Lo standard del Modello: codici HTTP semanticamente corretti, messaggi generici al client, dettaglio tecnico solo nel log.

```
// SPW: il dettaglio va nel log, non al client
} catch (PDOException $e) {
```



```

error_log('auth: ' . $e->getMessage()); // solo qui
http_response_code(500);
echo json_encode(['status' => 'error', 'message' => 'Errore interno. Rip
rova.']);
}

```

ATTENZIONE

Non rimandare l'eccezione al client DIS fa l'opposto: auth.php, users.php, participants.php rispediscono `$e->getMessage()` direttamente al client.

```

// DIS auth.php:48 (anti-pattern)
} catch (PDOException $e) {
    echo json_encode(['status' => 'error', 'message' => $e->getMessage()]);
    // leak dei dettagli DB
}

```

È information disclosure gratuita: un attaccante legge nomi di tabelle e colonne dai messaggi d'errore. La regola «parlante per l'utente, opaco per l'attaccante» non costa nulla, basta non saltarla.

✱
✱ ✱

12. Le azioni distruttive e pubbliche

L'ultimo fronte è il più scivoloso: cosa succede quando un'azione è protetta da login ma anche distruttiva, oppure è pubblica (nessun login) ma deve comunque difendersi dall'abuso.

12.1 Il reset a un clic: perché «gated» non basta

In DIS gli endpoint `reset_system.php` e `reset_votes.php` richiedono il login admin ma non hanno CSRF. Una POST cross-site verso `reset_system.php`, innescata mentre l'admin è loggato in un'altra scheda, cancella tutti i partecipanti, i voti e gli audio. L'unica mitigazione è il `SameSite` di default del cookie (non impostato, come visto al §4), che dipende dalla versione di PHP e non copre ogni caso.

ATTENZIONE

Perché un'azione gated ha comunque bisogno del CSRF «È protetta da login» e «è protetta dal CSRF» sono garanzie diverse. Il login dice chi sei; il CSRF dice se sei stato tu a volerlo. Un'azione distruttiva ha bisogno di entrambe: senza CSRF, è la sessione legittima dell'admin a essere usata contro di lui. E una `confirm` JavaScript («sei sicuro?») non sostituisce il CSRF, perché gira sul client e l'attaccante la salta.

12.2 Il backup just-in-time: la difesa che DIS ha e SR no

E qui arriva la sorpresa che rompe la scala. Lo stesso DIS che non ha CSRF sul reset fa una cosa che il flagship degli incidenti, SR, non fa: copia il database prima di toccarlo.

```

// public/api/reset_votes.php:18-21 - backup prima del distruttivo
$dbPath = __DIR__ . '/.data/database.sqlite';
if (file_exists($dbPath)) {
    copy($dbPath, __DIR__ . '/.data/backup_votes_' . date('Ymd_His') . '.sql

```

```
ite.bak');  
}  
$pdo->exec("DELETE FROM votes");
```

È esattamente la prevenzione che mancava a SR (al CAP 15 la chiamiamo «cura senza prevenzione»): il sito più debole sull'identità è l'unico a fare il backup giusto-in-tempo. Più sicuro non significa più completo su ogni punto.

12.3 Anti-frode di un'azione pubblica, e privacy

Il voto del festival è un'azione che non è autenticata: è il pubblico a votare. DIS la difende a strati, con una sola barriera reale, la `REMOTE_ADDR`/24h vista al box IP del §6. Il trattamento completo dell'anti-frode voto (master switch, cookie cosmetico, `vote_count` denormalizzato) vive al CAP 18; qui interessa solo un punto trasversale: come si conserva l'identità di chi compie un'azione pubblica.

CONSIGLIO

Votare in anonimato: hash invece di IP in chiaro (*ponte al CAP 20*) DIS salva `ip_address` e `user_agent` in chiaro nella tabella `votes`. Funziona per l'anti-doppio-voto, ma conserva dati personali senza necessità. SPW, per le reazioni anonime (CAP 20), ottiene la stessa garanzia anti-frode memorizzando solo `voter_hash = SHA256(IP + UA)`; il confronto regge, ma il dato personale non viene mai scritto. Due posture privacy opposte sulla stessa esigenza funzionale. Nota però che l'hash di SPW non è salato e usa input a bassa entropia (IP e UA): è anti-collisione, non anonimato crittografico forte, perché un IP candidato si può ancora verificare per forza brutta. Il confronto pieno tra le due filosofie (sanitizzazione write-time contro render-time, e identità anonima) è al CAP 20.

Sul versante anti-abuso, c'è un ultimo dettaglio: lo stesso `login_attempts` di SPW viene riusato come rate-limit a due strati per le reazioni (per-hash 20/min più solo-IP 30/min): il primo strato si aggira ruotando lo User-Agent, il secondo è l'argine vero. Anche qui il dettaglio è al CAP 20.

**
**

13. Il lato client e una lezione sui bot

L'area Admin in React è protetta da un `AdminLayout` che, nel suo `useEffect` principale, interroga `check_auth`: se il server risponde 401, il client distrugge lo stato locale e reindirizza al login, così non lampeggia contenuto sensibile. La sidebar e le rotte si generano in base al `role` ricevuto dal server (admin contro editor in SR e DIS).

NOTA

Il client non è la difesa. Il logout client-side e il check 401 sono esperienza utente, non sicurezza: nascondono ciò che non devi vedere, ma è il gate server-side (§2) a impedirti di toccarlo. La differenza tra «logout sul client» e l'invalidazione vera via `session_version` (§7) è esattamente questa. L'architettura completa del pannello ad-

min (le tre dashboard, le tre architetture di guardia, il posizionamento dei backup) è al CAP 14.

C'è infine una lezione che viene da un incidente reale e resta pertinente qui, anche se il caso completo è migrato altrove. Nel febbraio 2026 Runtime Radio è stato sommerso da bot che simulavano i crawler dei social (Telegram, Facebook, X) per colpire l'entry-point SEO in PHP, che a ogni richiesta interrogava il database. Lo User-Agent dei bot è falsificabile.

ATTENZIONE

Lo User-Agent non è un gatekeeper Non si prendono mai decisioni di sicurezza in base allo User-Agent, perché si falsifica in un campo header. Lo si può usare per ottimizzare (servire una cache ai bot riconosciuti), mai come barriera d'accesso. Il racconto completo dell'attacco DDoS-da-bot e la soluzione (cache statica precompilata, percorso bot separato da quello umano) sono al CAP 11, perché il vettore è proprio l'entry-point SEO. Qui resta solo la massima.

In sintesi

La sicurezza è la lente che smentisce l'intuizione «più grande uguale più robusto». SPW, che non è il sito più ingegnerizzato, ha il perimetro più maturo. SR, il più ricco, lascia tre buchi reali: cookie senza `Secure`, niente anti-fixation, rate-limit che si aggira. DIS, il grado zero, porta comunque due idee che agli altri mancano, l'anti-frode pubblica robusta e il backup pre-distruttivo. Letta come una scala di sottrazione, ogni difesa insegna due cose insieme: cosa fa, e cosa si rompe quando la toglie.

IMPORTANTE Il Canone

- Sessioni con cookie `HttpOnly + SameSite=Strict + Secure` su HTTPS; password con `password_hash()`.
- Token CSRF su tutte le mutazioni: un `confirm()` non è una difesa CSRF.
- Autorizzazione per **ruolo** (`isAdmin`), non solo per login (`isLoggedIn`).
- Lockout brute-force misurato su un IP affidabile, non su header controllati dal client (`X-Forwarded-For`).
- `session_version` per invalidare le sessioni al cambio password; nessuna credenziale di default nel codice.

Prossimo Capitolo: SEO Pre-rendering con PHP Entry-Point. Il motore SEO invisibile che trasforma una SPA in un sito indicizzabile, e il vettore dell'attacco DDoS-da-bot del febbraio 2026.